

LAB Assignment No. 10

AI-Based Face Detection System using ESP32, Python OpenCV, and ThingSpeak Cloud

The objective of this lab assignment is to design and implement an IoT-based intelligent monitoring system in which:

- A laptop camera captures live video using Python and OpenCV.
- Face detection is performed using Artificial Intelligence (Computer Vision).
- Detection status is sent to ESP32 over WiFi.
- ESP32 controls an LED:
 - LED ON → Face detected
 - LED OFF → No face detected
- Detection data is uploaded to the cloud using ThingSpeak.

Learning Objectives

Students learn integration of:

- Computer Vision
- ESP32
- IoT Cloud Platforms
- Wireless Communication
- Real-Time Monitoring

Learning Outcomes

After completing this assignment, students will be able to:

1. Understand IoT system integration.
2. Use Python OpenCV for face detection.
3. Interface ESP32 with web requests.
4. Control hardware using wireless communication.
5. Send sensor/status data to ThingSpeak cloud.

6. Develop real-time AI-based IoT applications.

Required Software

Software	Purpose
Visual Studio Code	Python programming
Python	Face detection
Wokwi	ESP32 simulation
OpenCV	Face detection
Arduino IDE	ESP32 programming

Required Hardware

Component	Quantity
ESP32 Dev Board	1
LED	1
220Ω Resistor	1
Breadboard	1
Jumper Wires	As required
Laptop with Webcam	1

Theory Background

1. Face Detection using OpenCV

OpenCV is a Computer Vision library used to process images and videos. Haar Cascade classifier is used to detect human faces from webcam video frames.

2. ESP32 WiFi Communication

ESP32 acts as a web server and receives HTTP requests from Python code.

Example:

```
http://ESP32_IP/update?value=1
```

Where:

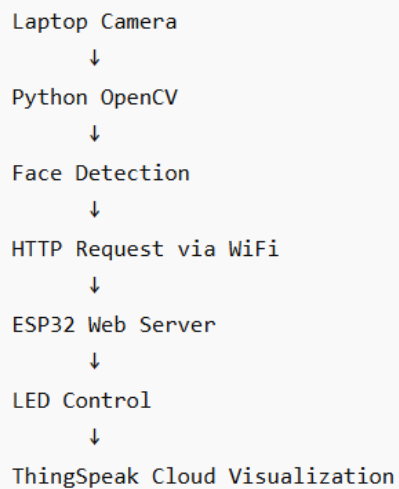
- value=1 → Face detected
- value=0 → No face detected

3. ThingSpeak Cloud

ThingSpeak is an IoT cloud platform used to visualize sensor data and monitoring information.

The ESP32 uploads detection status to ThingSpeak channel fields.

System Architecture



Circuit Diagram Connections

ESP32 Pin	Component
GPIO2	LED Positive
GND	LED Negative

Wokwi Simulation Steps

1. Open [Wokwi](#)
2. Create New ESP32 Project
3. Add:
 - ESP32
 - LED
 - Resistor
4. Connect:
 - GPIO2 → Resistor → LED → GND
5. Paste ESP32 code
6. Run simulation

ESP32 code:

```
#include <WiFi.h>
#include <WebServer.h>
#include <ThingSpeak.h>

const char* ssid = "Wokwi-GUEST";
const char* password = "";

WiFiClient client;
WebServer server(80);

#define LED 17

unsigned long channelID = 3375895;
const char* writeAPI = "A4UIZeqHTMFZC5RE";

unsigned long lastUpdateTime = 0;
int lastState = -1;

void handleFace() {
```

```

if (!server.hasArg("value")) {
  server.send(400, "text/plain", "Missing value");
  return;
}

int val = server.arg("value").toInt();

Serial.println("=====");
Serial.print("HTTP Request Received: ");
Serial.println(val);

if (val == 1) {
  digitalWrite(LED, LOW);
  Serial.println("STATUS: FACE DETECTED → LED ON");
} else {
  digitalWrite(LED, HIGH);
  Serial.println("STATUS: NO FACE → LED OFF");
}

// Prevent repeated same logs spam
if (val != lastState) {
  Serial.print("STATE CHANGED → ");
  Serial.println(val);
  lastState = val;
}

// ThingSpeak update
if (millis() - lastUpdateTime > 15000) {
  ThingSpeak.setField(1, val);

  int response = ThingSpeak.writeFields(channelID, writeAPI);

  Serial.print("ThingSpeak Response: ");
  Serial.println(response);

  lastUpdateTime = millis();
}

server.send(200, "text/plain", "OK");
}

void setup() {
  Serial.begin(115200);

  pinMode(LED, OUTPUT);

```

```
digitalWrite(LED, HIGH);

Serial.println("Connecting to WiFi...");
WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
}

Serial.println("\nWiFi Connected!");
Serial.print("IP: ");
Serial.println(WiFi.localIP());

ThingSpeak.begin(client);

server.on("/update", handleFace);
server.begin();

Serial.println("Server Started");
}

void loop() {
  server.handleClient();
}
```

Python OpenCV Face Detection Code

Install libraries:

```
pip install opencv-python requests
```

Required Python Libraries

Open terminal in VS Code and install:

pip install opencv-python

pip install numpy

Optional (for advanced projects later):

pip install mediapipe

pip install face-recognition

Python code

```
import cv2
import requests

ESP32_URL = "http://10.10.0.2/update"
THINGSPEAK_KEY = "A4UIZeqHTMFZC5RE"

face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
)

cap = cv2.VideoCapture(0)

last_value = -1 # IMPORTANT: prevents repeated requests

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    value = 1 if len(faces) > 0 else 0

    if value == 1:
        label = "FACE DETECTED"
    else:
        label = "NO FACE"

    print(label)

# 📡 Send only when state changes
```

```

if value != last_value:
    try:
        requests.get(f"{ESP32_URL}?value={value}", timeout=2)
    except Exception as e:
        print("ESP32 Error:", e)

    try:
        requests.get(
            f"https://api.thingspeak.com/update?api_key={THINGSPEAK_KEY}&field1={value}",
            timeout=2
        )
    except Exception as e:
        print("ThingSpeak Error:", e)

    last_value = value

# Draw face box
for (x, y, w, h) in faces:
    cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

cv2.putText(frame, label, (20, 50),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

cv2.imshow("AloT Face Detection", frame)

# Quit key
if cv2.waitKey(1) & 0xFF in [ord('q'), ord('Q')]:
    break

cap.release()
cv2.destroyAllWindows()

```


Expected Output

When Face is Detected

Python Console

```
Face Detected
```

ESP32 Serial Monitor

```
FACE DETECTED -> LED ON  
ThingSpeak Updated
```

Hardware

- LED turns ON

ThingSpeak

- Field value = 1

When Face is NOT Detected

Python Console

```
No Face Detected
```

ESP32 Serial Monitor

```
NO FACE DETECTED -> LED OFF  
ThingSpeak Updated
```

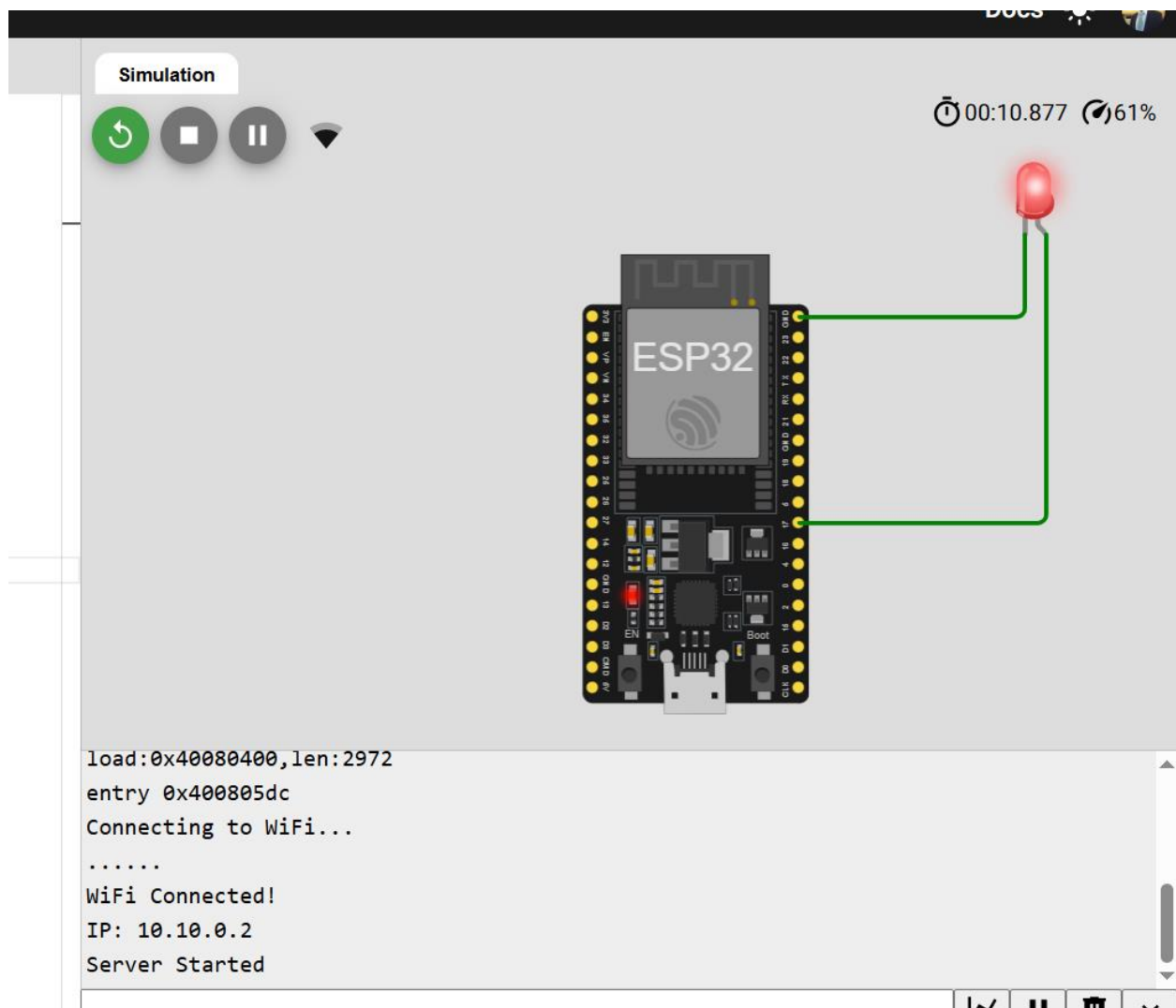
Hardware

- LED turns OFF

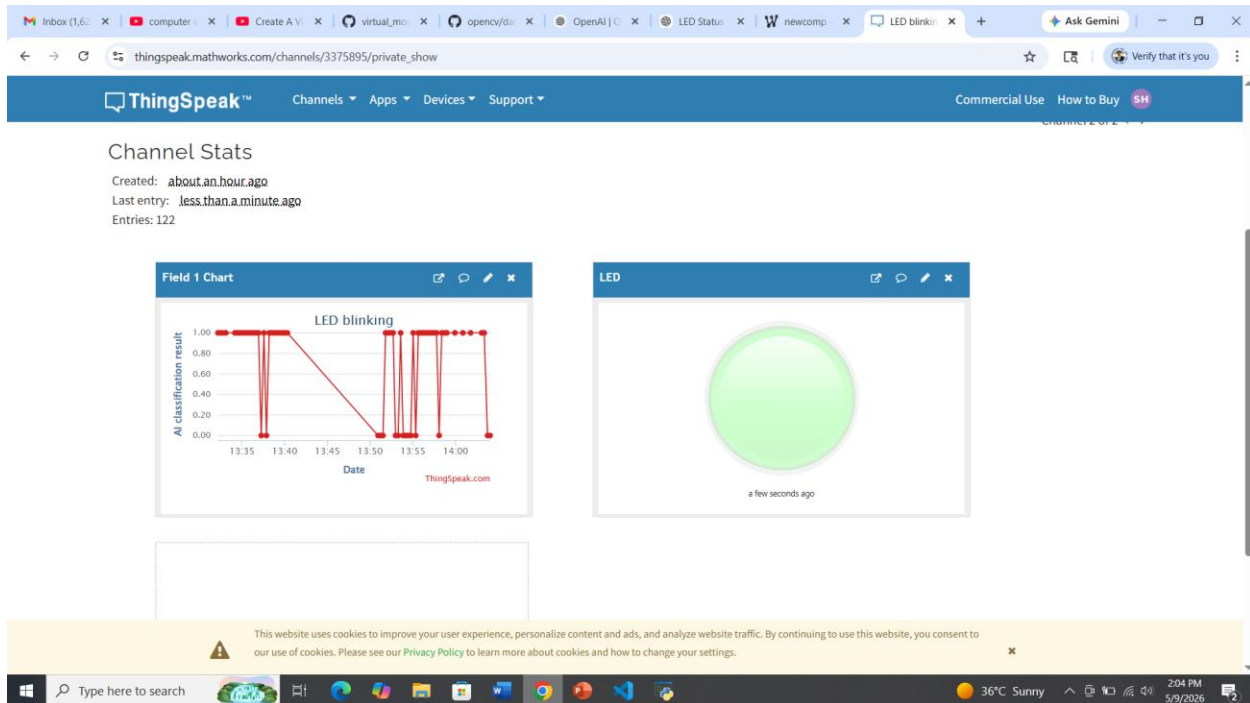
ThingSpeak

- Field value = 0

Screenshots Requirement of this Lab Assignment



Face is not detected



```
File Edit Selection View Go ... < -> compVision Sign In
```

EXPLORER

- COMPVISION
- OUTLINE
- TIMELINE

main3.py > ...

```
1 import cv2
2 import requests
3
4 ESP32_URL = "http://10.10.0.2/update"
5 THINGSPEAK_KEY = "A4UIZEQHTMFZC5RE"
6
7 face_cascade = cv2.CascadeClassifier(
8     cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
9 )
10
11 cap = cv2.VideoCapture(0)
12
13 last_value = -1 # IMPORTANT: prevents repeated requests
14
15 while True:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

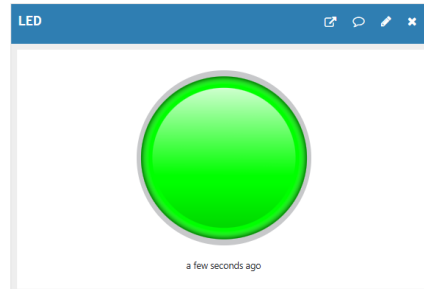
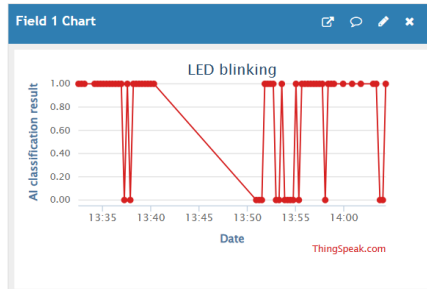
```
(venv) PS C:\Users\Syed Hamedoon\compVision> python -u "c:\Users\Syed Hamedoon\compVision\main3.py"
NO FACE
NO FACE
FACE DETECTED
FACE DETECTED
NO FACE
```

Channel Stats

Created: [about an hour ago](#)

Last entry: [less than a minute ago](#)

Entries: 123



Review Questions

Tasks for Students

Students must perform the following tasks:

Task 1

Create ThingSpeak channel with:

- Field 1 = Face Detection Status
-

Task 2

Design ESP32 circuit in Wokwi.

Task 3

Write and upload ESP32 code.

Task 4

Develop Python OpenCV face detection program.

Task 5

Integrate Python with ESP32 using HTTP requests.

Task 6

Verify:

- LED ON/OFF operation
- Serial Monitor output
- ThingSpeak cloud updates

Screenshots Required in Report

Students must include:

1. Wokwi circuit diagram
2. ESP32 serial monitor
3. Python execution window
4. Face detection output
5. LED ON condition
6. LED OFF condition
7. ThingSpeak dashboard graphs

Viva Questions

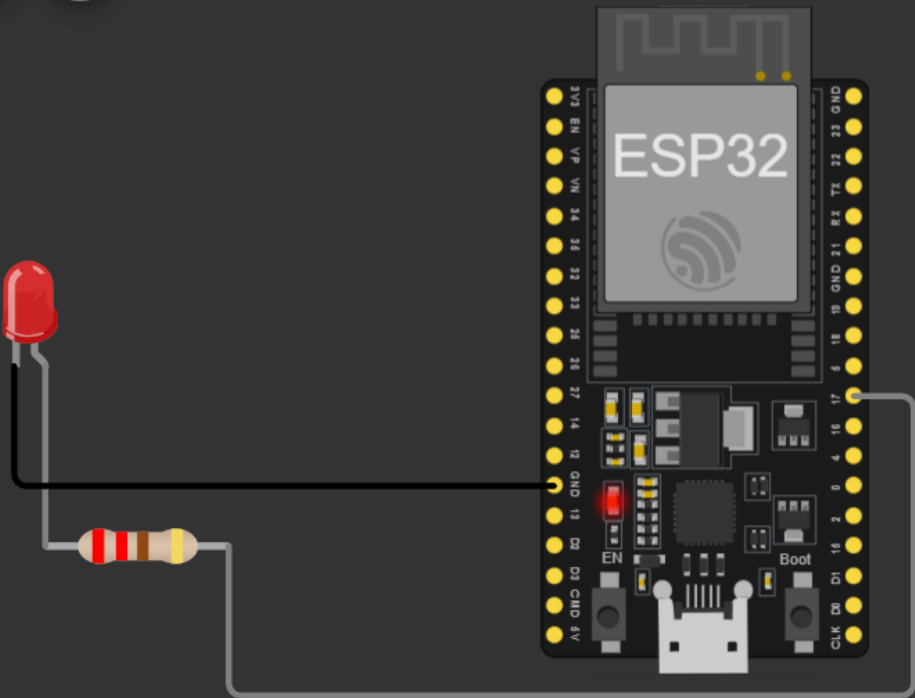
1. What is IoT?
2. What is the role of ESP32?
3. Why is OpenCV used?
4. What is Haar Cascade classifier?
5. How does HTTP communication work?
6. Why is ThingSpeak used?
7. What happens when no face is detected?
8. Explain WiFi communication in ESP32.
9. Difference between cloud computing and IoT?
10. What are real-world applications of this project?

Docs  

Simulation

01:29.529 42%



NO FACE

Face Detection

NO FACE

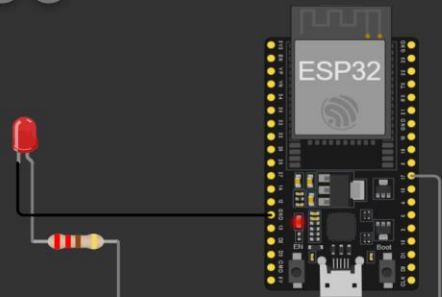
24 Serial.print(".");
25 }

Docs  

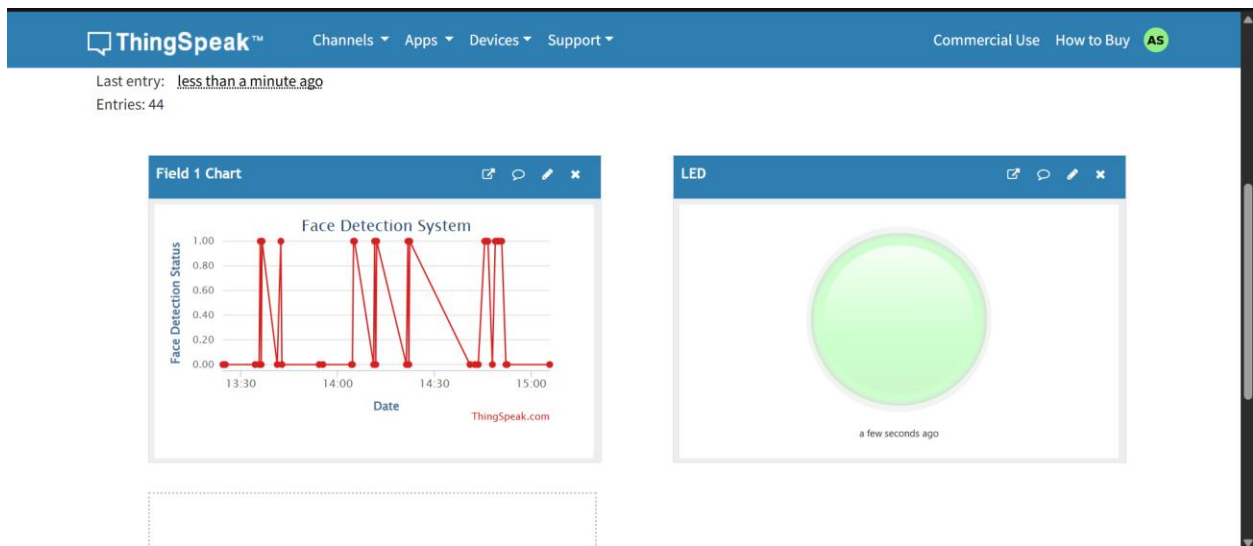
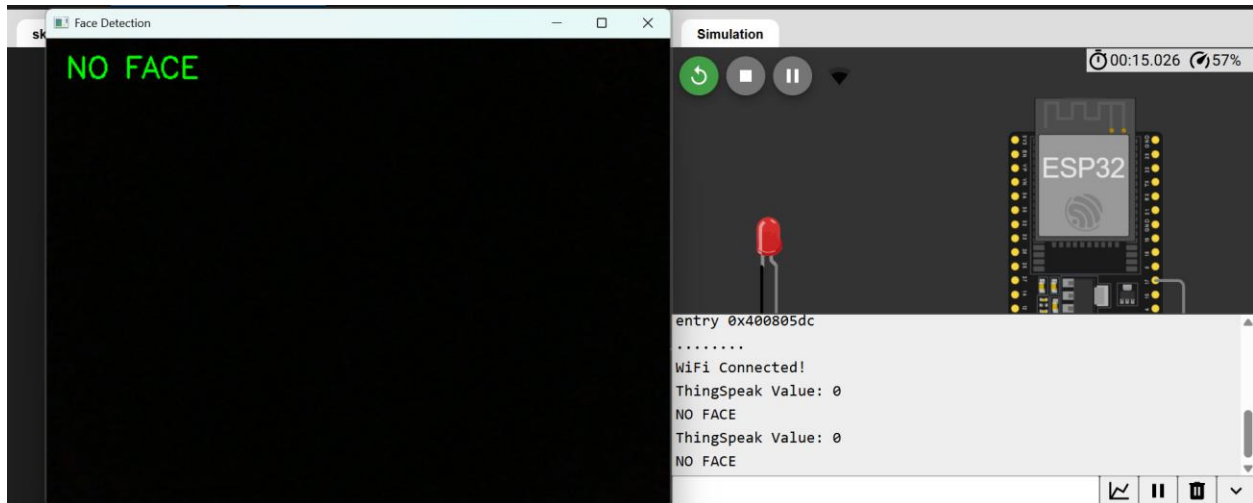
Simulation

01:36.912 40%



NO FACE



C++ CODE:

```

1  #include <WiFi.h>
2  #include <HTTPClient.h>
3  #include <ThingSpeak.h>
4
5  const char* ssid = "Wokwi-GUEST";
6  const char* password = "";
7
8  unsigned long channelID = 3376654;
9  const char* readAPI = "VSOXPZQHGRY4IRXA";
10
11 #define LED 17
12
13 WiFiClient client;
14
15 void setup() {
16     Serial.begin(115200);
17
18     pinMode(LED, OUTPUT);
19
20     WiFi.begin(ssid, password);
21
22     while (WiFi.status() != WL_CONNECTED) {
23         delay(500);
24         Serial.print(".");
25     }

```

```

25     }
26
27     Serial.println("\nWiFi Connected!");
28
29     ThingSpeak.begin(client);
30 }
31
32 void loop() {
33
34     int value = ThingSpeak.readFloatField(channelID, 1, readAPI);
35
36     Serial.print("ThingSpeak Value: ");
37     Serial.println(value);
38
39     if (value == 1) {
40         digitalWrite(LED, HIGH);
41         Serial.println("FACE DETECTED");
42     } else {
43         digitalWrite(LED, LOW);
44         Serial.println("NO FACE");
45     }
46
47     delay(3000);
48 }

```

Python Code:

```

face_detection.py 2 x
face_detection.py > ...
1  import cv2
2  import requests
3  import time
4  import sys
5
6  # ThingSpeak
7  THINGSPEAK_KEY = "VS0XPZQHGRY4IRXA"
8
9  last_thingspeak_time = 0
10 last_value = -1
11
12 face_cascade = cv2.CascadeClassifier(
13     cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
14 )
15
16 cap = cv2.VideoCapture(0)
17
18 print("Camera started... Press Q to quit")
19
20 while True:
21     ret, frame = cap.read()
22     if not ret:
23         continue
24
25     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
26
27     faces = face_cascade.detectMultiScale(gray, 1.3, 5)
28
29     value = 1 if len(faces) > 0 else 0

```

Screen Reader Optimized | Ln 70, Col 24 | Spaces: 4 | UTF-8 | CRLF | {} Python | 3.14.2 | venv (3.14.2) | Go Live | Prettier

```

30     label = "FACE DETECTED" if value == 1 else "NO FACE"
31
32     if value != last_value:
33         print("State:", label)
34
35         # ☒ ONLY ThingSpeak (FIXED)
36         if time.time() - last_thingspeak_time >= 15:
37             try:
38                 r = requests.get(
39                     "https://api.thingspeak.com/update",
40                     params={
41                         "api_key": THINGSPEAK_KEY,
42                         "field1": value
43                     },
44                     timeout=5
45                 )
46                 print("ThingSpeak Updated:", r.text)
47                 last_thingspeak_time = time.time()
48             except Exception as e:
49                 print("ThingSpeak Error:", e)
50
51     last_value = value
52
53     for (x, y, w, h) in faces:
54         cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
55
56     cv2.putText(frame,
57                 f"{label}",
58                 (20, 40),

```

```

58         (20, 40),
59         cv2.FONT_HERSHEY_SIMPLEX,
60         1,
61         (0, 255, 0),
62         2)
63
64     cv2.imshow("Face Detection", frame)
65
66     if cv2.waitKey(1) & 0xFF == ord('q'):
67         break
68
69 cap.release()
70 cv2.destroyAllWindows()

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE powershell + - [] [X] ... | [] [X]

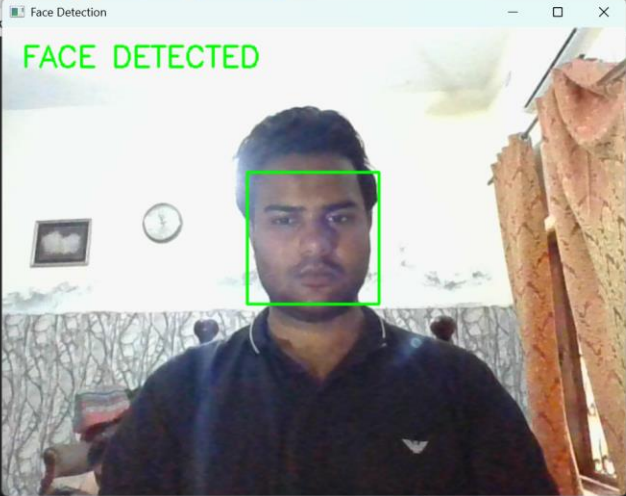
(venv) PS C:\Users\hp\OneDrive\Documents\FaceDetectionProject> py face_detection.py
Camera started... Press Q to quit
State: NO FACE
ThingSpeak Updated: 44
State: FACE DETECTED
ThingSpeak Updated: 45
State: NO FACE
State: FACE DETECTED

Screen Reader Optimized Ln 70, Col 24 Spaces: 4 UTF-8 CRLF Python 3.14.2 venv (3.14.2) Go Live Prettier

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
(venv) PS C:\Users\hp\OneDrive\Documents\FaceDetectionProject> py face_detection.py
ThingSpeak Updated: 49
State: FACE DETECTED
State: NO FACE
State: FACE DETECTED
State: NO FACE
State: FACE DETECTED
State: NO FACE
```

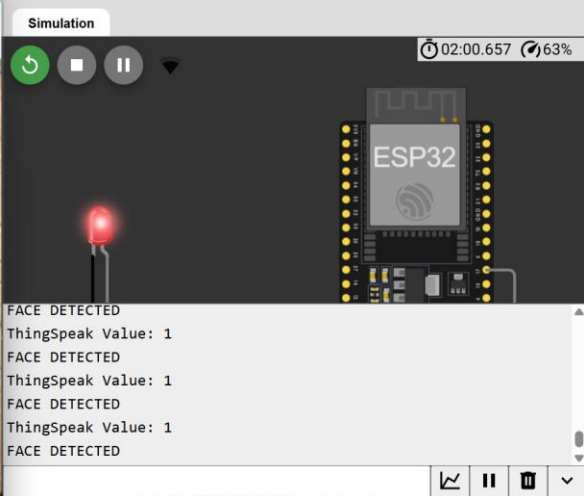
WOKWI SAVE SHARE Face Detection System Docs

Face Detection



Simulation

02:00.657 63%



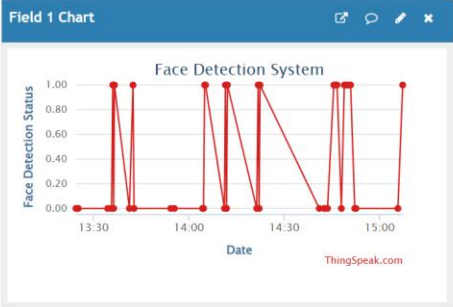
FACE DETECTED
ThingSpeak Value: 1
FACE DETECTED
ThingSpeak Value: 1
FACE DETECTED
ThingSpeak Value: 1
FACE DETECTED

ThingSpeak™ Channels Apps Devices Support Commercial Use How to Buy AS

Last entry: less than a minute ago
Entries: 45

Field 1 Chart

Face Detection System



ThingSpeak.com

LED



a few seconds ago